HARDWARE EXCEPTION VS. INTERRUPT REFERENCE GUIDE

Systems Programming & Architecture Technical Template

1. ARCHITECTURAL DEFINITIONS & CORE CLASSIFICATIONS

In modern CPU architectures, unexpected or asynchronous changes in control flow are collectively managed through a generalized mechanism known as **Asynchronous Flow Control** or **Exception Handling**. However, system design dictates a strict operational distinction between an **Interrupt** and an **Exception** based on their origin, timing synchronization, and structural purpose.

Interrupt (Hardware/External)

An asynchronous signal generated by external hardware subsystems completely independent of the currently executing CPU instruction stream. They alert the processor to external events requiring deterministic response windows without polling.

Exception (Software/Internal)

A synchronous event generated internally by the CPU core execution unit as a direct result of executing a specific instruction. Exceptions signal an error state, resource deficiency, or an explicit request for privilege escalation.

Vector Parameter	Hardware Interrupts	Software / Processor Exceptions
Source	External to the processor silicon core (e.g., PCIe controllers, timers, GPIO pins, DMA controllers, PMICs).	Internal to the execution pipeline (e.g., ALU, MMU, Instruction Decoder).
Timing Relation	**Asynchronous:** Can strike at any arbitrary clock cycle. Independent of the current program counter (***PC***).	**Synchronous:** Bound exactly to the execution phase of the offending or invoking instruction. Reproducible.
Typical Root Causes	• Inbound network packets available • Hardware timer period expiration • Peripheral buffer full/empty status • External sensor threshold trigger	• Fatal conditions (Faults: Division by zero) • Memory management flags (Page Faults) • Intentional trap vectors (Syscalls) • Architectural violations (Illegal opcode)
Program Counter Position	Points to the *next* instruction that was scheduled to execute when the interrupt line asserted.	Points to either the exact instruction causing the fault, or the immediate following instruction (Trap).

2. SYSTEM EXECUTION MECHANICS & VECTOR WORKFLOW

Regardless of the triggering source, both events enforce a microarchitectural pivot in execution state. The hardware sequence forces an atomic transition from non-privileged thread space to privileged handler space.

Unified Handling Workflow Pipeline:

1. **Atomic Boundary Capture (State Saving):** Upon recognizing the vector assertion, the pipeline completes its current architectural instruction (for interrupts) or halts it (for faults). The CPU automatically pushes fundamental context records—such as the Program Counter (***PC***), Stack Pointer (***SP***), and Processor Status Register (***PSR***)—to a dedicated privilege stack or hardware shadow registers.

2. **Vector Identification & Table Routing:** The CPU looks up the assigned numerical entry index within the system *Vector Table* (e.g., Interrupt Vector Table or Exception Vector Table) based on hardware lines or internal status IDs. The base memory address of this table is tracked inside a dedicated control register (e.g., `VBAR` in ARM or `IDTR` in x86).
3. **Privilege Escalation & Handler Branching:** The CPU switches execution modes (e.g., switching from User/Thread mode to Supervisor/Handler mode), establishes the new stack boundary, and loads the target handler function address into the ***PC***.
4. **Software PUSH Execution:** The software Interruption Service Routine (ISR) or Exception Handler saves any remaining general-purpose working registers (***R0-R12***, `RAX`, etc.) onto the active stack to ensure complete context safety before execution begins.
5. **Payload Execution:** The active routine executes. For interrupts, it clears peripheral flags. For exceptions, it fixes memory frames (Page Fault resolution) or wraps up kernel requests.
6. **Context Restoration & Return (Atomic Pop):** The software pops the general-purpose registers, then invokes an architecturally specific return instruction (e.g., `IRET` or `BX LR`). This atomically restores the original processor state, changes the privilege level back to the previous context, and drops the original ***PC*** back into place.

3. CPU-SPECIFIC ARCHITECTURAL IMPLEMENTATIONS

Low-level embedded software engineers must manage distinct context-switching rules across different CPU families. Below are the implementation requirements for ARM Cortex-M and x86-64 architectures.

A. ARM Cortex-M Architecture (Hardware Auto-Stacking)

ARM Cortex-M processors implement a hybrid hardware-software context-saving model to minimize interrupt latency. This mechanism, known as **hardware auto-stacking**, automatically saves core registers onto the active stack (MSP or PSP) in parallel with fetching the vector address.

```
+-------------------------------------------------------------+
|  Hardware Automated Save Frame (8 Words Stacked by CPU)     |
|  [ xPSR ] -> [ PC ] -> [ LR ] -> [ R12 ] -> [ R3 ... R0 ]   |
+-------------------------------------------------------------+
|  Software Manual Save Frame (Required for Context Switch)   |
|  [ R11 -> R4 ] (Pushed inside OS Scheduler / ISR Wrapper) |
+-------------------------------------------------------------+
```

Because the hardware automatically stacks the scratch registers (`R0-R3`, `R12`, `LR`, `PC`, `xPSR`), standard C functions can serve directly as ISRs without assembly wrappers. If a context switch is required, the OS scheduler must manually push the remaining callee-saved registers (`R4-R11`).

B. x86-64 Architecture (Interrupt Stack Frame & Privilege Gates)

In x86-64 long mode, the CPU switches stacks using the Interrupt Stack Table (IST) found within the Task State Segment (TSS). When an interrupt or exception bypasses user privilege constraints, the CPU pushes a fixed hardware frame onto the target stack.

```
+-------------------------------------------------------------+
|  x86-64 Hardware Generated Interruption Frame               |
|  [SS] -> [RSP] -> [RFLAGS] -> [CS] -> [RIP] -> [Error Code]|
+-------------------------------------------------------------+
|  Software Manual Assembly Save Frame                        |
```

```
|  [RAX, RCX, RDX, RSI, RDI, R8-R11 ... Remaining Registers] |
+-----------------------------------------------------------+
```

Critical architectural variant: For specific processor exceptions (such as Page Faults `#PF` or General Protection Faults `#GP`), the CPU appends an explicit **Error Code** onto the stack below the Instruction Pointer (`RIP`). This error code must be explicitly popped from the stack before executing an `IRETQ` instruction to prevent stack misalignment and subsequent system crashes.